

Relatório de Desenvolvimento: Retail Vision Intelligence System

Inês Alves n55182

Licenciatura em Inteligência Artificial e Ciência de Dados (LIACD)

Trabalho Prático #2 - Interação com Modelos de Grande Escala

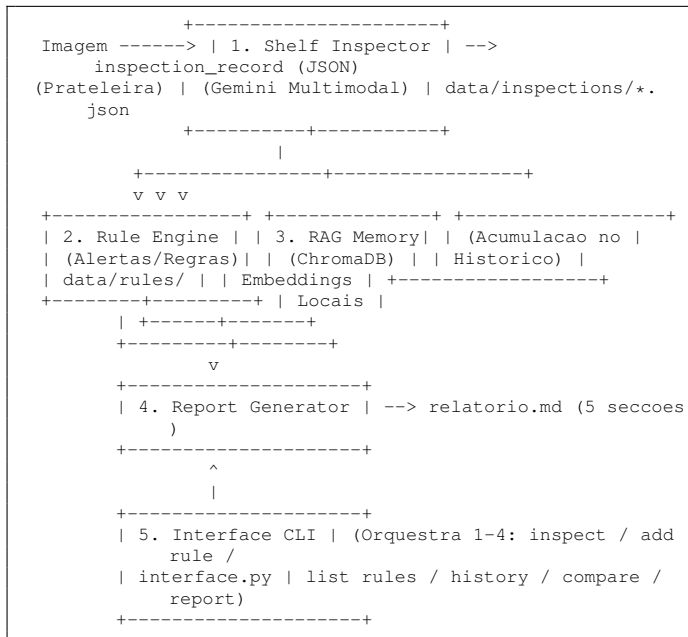
Ano letivo 2025/2026

I. INTRODUÇÃO E ARQUITETURA DO SISTEMA

A. Visão Geral

O sistema desenvolvido implementa um pipeline de inspeção automática de prateleiras de supermercado. A solução proposta combina análise visual multimodal baseada em Grandes Modelos de Linguagem (LLMs), um motor de regras configurável em linguagem natural, uma componente de memória semântica de longo prazo através de Geração Aumentada por Recuperação (RAG) e a geração automatizada de relatórios gerenciais. Todo este ecossistema é orquestrado por uma interface unificada de linha de comandos (CLI).

O sistema é composto por cinco componentes principais que comunicam de forma assíncrona e integrada através de um formato de dados comum — o `inspection_record` em formato JSON — e de ficheiros persistidos no diretório local `data/`:



B. Decisões de Design de Alto Nível

- **Campos de Sistema vs. Campos do Modelo:** Os metadados críticos (`inspection_id`, `timestamp`, `image_path` e `zone_id`) são estritamente gerados e injetados pelo sistema, nunca sendo aceites a partir da resposta direta do LLM. Esta abordagem garante a integridade

referencial dos dados, mitigando os efeitos de eventuais alucinações do modelo.

- **Cache Determinístico por MD5:** Implementou-se um mecanismo de cache local baseado no algoritmo MD5 do conteúdo binário das imagens. A chave de indexação combina o hash da imagem, a estratégia de prompt e o identificador da zona (`cache/<hash>_<estratégia>_<zone_id>.json`), prevenindo chamadas redundantes à API e otimizando a quota diária gratuita.
- **Mecanismo de Fallback Gracioso:** Qualquer anomalia no pipeline (exaustão de quotas, falhas de rede ou respostas malformadas que quebrem o parser de JSON) é interceptada pela função `analyze_image`, que gera um `inspection_record` estruturalmente válido com `overall_status="unknown"` e a flag `_fallback=true`. Isto assegura a resiliência e a continuidade do fluxo operacional.
- **Separação de Regras por Validação:** O componente de conversão de regras isola assincronamente as diretivas consideradas ambíguas (`validation.is_valid=false`), mantendo-as disponíveis para consulta e auditoria, mas excluindo-as do motor de inferência ativo (`active_only=True`).
- **Processamento RAG Local:** A indexação e a pesquisa semântica são executadas com recursos a embeddings locais e à base de dados vetorial ChromaDB, eliminando custos de chamadas de rede para a recuperação, reservando o uso do LLM apenas para a síntese textual final.
- **Geração de Relatórios Determinística:** O gerador de relatórios prescinde de chamadas à API de inferência, operando exclusivamente através de templates baseados nos dados estruturados consolidados pelas componentes anteriores.

II. CONSTRUÇÃO E TRATAMENTO DO DATASET (FASE 1)

A. Aquisição das Imagens em Streaming

Como fundação do dataset, utilizou-se o subconjunto do SKU-110K (Goldman et al., CVPR 2019), composto por imagens de cenários reais de retalho. Dado que o arquivo original compreende 12.2 GB de dados, inviabilizando o download integral para o âmbito deste projeto, desenvolveu-se o script `scripts/download_sku110k_subset.py`.

O script abre uma ligação HTTP em modo streaming (`stream=True`) e processa o arquivo `.tar.gz` de forma incremen-

tal na memória via `tarfile.open(fileobj=resp.raw, mode="r|gz")`. Filtram-se e extraem-se individualmente apenas as imagens formatadas em .jpg, .jpeg ou .png pertencentes ao subconjunto de treino (train), cessando a execução imediatamente após atingir o limite estipulado pelo parâmetro `-n`. A execução padrão seguiu a seguinte sintaxe:

```
python scripts/download_data.py --n 600 --out data/images/
raw_data
```

Nota de Engenharia sobre Robustez: Durante o processo de ingestão, ocorreu uma interrupção motivada por um erro de timeout do socket (`ReadTimeoutError`) na ligação ao servidor S3 da Trax ao fim de 50 imagens. O sistema recuperou de forma bem-sucedida recorrendo ao parâmetro `-skip`, permitindo saltar as N correspondências já persistidas em disco e completar o lote de 600 imagens sem duplicações.

B. Categorização Heurística: Métrica de Desordem Visual

Para operacionalizar a distribuição das 600 imagens sem incorrer em custos com APIs de visão computacional, desenhou-se uma métrica heurística designada por *clutter score*. O algoritmo avalia as características morfológicas e cromáticas dos pixéis através da seguinte formulação matemática:

$$\text{score} = 0.7 \times \sigma(\text{deteção de contornos}) + 0.3 \times \sigma(\text{imagem RGB}) \quad (1)$$

O cálculo segue o seguinte protocolo por imagem:

- 1) **Normalização:** Conversão da imagem para escala de cinzentos e redimensionamento para a resolução estática de 256×256 pixéis para homogeneizar o custo computacional.
- 2) **Extração de Arestas:** Aplicação do filtro de deteção de contornos `PIL.ImageFilter.FIND_EDGES`, isolando as transições de intensidade que delimitam produtos, faces de embalagens e etiquetas.
- 3) **Cálculo da Densidade Visual:** Computação do desvio-padrão dos pixéis resultantes (σ_{edges}), assumindo que valores elevados correlacionam-se com maior densidade e desorganização de itens na prateleira.
- 4) **Variabilidade Cromática:** Computação do desvio-padrão dos canais RGB originais (σ_{cor}) para mensurar a diversidade de cores.
- 5) **Ponderação:** Fusão linear dos valores (70% contornos, 30% cor) para obter o indicador final.

A premissa teórica valida-se pelo facto de prateleiras vazias ou alinhadas gerarem amplas superfícies uniformes (scores baixos), ao passo que gôndolas saturadas ou em desordem induzem transições frequentes e dispersas (scores altos).

C. Distribuição e Análise Estatística dos Intervalos

As imagens foram ordenadas com base no seu score e segmentadas em buckets contíguos através do script `scripts/categorize_data.py`:

- **normal (150 imagens):** Amostras com as 150 métricas mais baixas.

- **dirty (80 imagens):** Amostras com as 80 métricas mais elevadas.
- **ambiguous (70 imagens):** Uma janela central extraída em torno da mediana restante, representando os casos limítrofes de difícil classificação unívoca.
- **leftover_for_synthetic (300 imagens):** Excedente não aproveitado nesta fase.

A tabela seguinte detalha os intervalos numéricos obtidos e os caminhos de armazenamento:

TABLE I
DISTRIBUIÇÃO DAS IMAGENS POR CATEGORIA (*clutter score*)

Cat.	N.º	Intervalo Score	Diretório	Método Seleção
Normal	150	48.15–66.59	data/images/normal	Heurística desordem (score baixo)
Suja (dirty)	80	75.93–83.09	data/images/dirty	Heurística desordem (score alto)
Ambígua	70	70.50–72.18	data/images/ambiguous	Heurística desordem (faixa central)
Leftover	300	66.68–75.90	data/images/leftover	Excluído (redundante face pools)

Os intervalos empíricos confirmam a consistência da métrica, evidenciando faixas estritamente crescentes e sem sobreposições espúrias entre categorias consecutivas (*normal* < *ambiguous* < *dirty*). A curtose observada na categoria *ambiguous* (≈ 1.68 pontos de amplitude) ratifica o seu comportamento como zona de fronteira. Efetuou-se uma validação visual por amostragem para garantir a precisão macro da categorização.

D. Geração das Categorias Semânticas Especiais

As classes *empty* (ausência total ou parcial de stock) e *planogram_violation* (rutura de alinhamento, danos ou falta de sinalética de preço) foram derivadas a partir de um pool secundário independente de 1500 imagens candidatos em `data/images/pool_candidates/`. O processamento envolveu uma abordagem híbrida dividida em dois scripts:

1) Triagem Híbrida (scripts/*scan_pool_for_categories.py*):

- **Pré-filtro:** Ordenação do pool e pré-seleção dos 40% mais uniformes como potenciais candidatos a *empty* e dos 40% mais caóticos como potenciais *planogram_violation*.
- **Validação Semântica:** Submissão das imagens pré-filtradas ao modelo Gemini via `src/shelf_inspector.py` (Estratégia B). Uma imagem era classificada em *empty* se o modelo detetasse uma anomalia do tipo *empty_shelf* cobrindo $\geq 8\%$ da área útil; e em *planogram_violation* se reportasse instâncias de *damaged*, *misaligned*, *wrong_product* ou *label_missing*. Os registos de inspeção JSON foram gravados em `data/inspections/_scan/` para auditoria.

2) **Complementação Heurística (scripts/select_empty_violation.py)**: Devido ao esgotamento das quotas diárias gratuitas da API durante a execução do processo anterior, este script secundário foi acionado para completar as quotas remanescentes até atingir o patamar estipulado de 100 imagens por classe. O preenchimento baseou-se estritamente nas pontuações de clutter score, selecionando de forma determinística os elementos de menor score para empty e de maior score para planogram_violation, salvaguardando a exclusão de ficheiros previamente validados.

A proveniência e o método de seleção individualizado foram consolidados no ficheiro data/images/manifest.csv através do script scripts/build_manifest.py, assegurando a rastreabilidade total do dataset final de 500 imagens.

E. Origem e Licenciamento dos Dados

A totalidade das imagens utilizadas no dataset (incluindo as categorias normal, dirty, ambiguous, leftover, empty e planogram_violation) deriva exclusivamente do subconjunto de treino do SKU-110K [1], disponibilizado pelos autores (Goldman et al., Bar-Ilan University / Trax Retail / CVPR 2019) em https://github.com/eg4000/SKU110K_CVPR19. O repositório oficial do dataset declara explicitamente que "the dataset is provided for the exclusive use by the recipient and solely for academic and non-commercial purposes", pelo que a sua utilização neste projeto se enquadra estritamente no âmbito académico e não comercial para o qual foi cedido, não tendo sido aplicada qualquer redistribuição ou uso fora deste contexto.

Não foram introduzidas imagens de outras fontes (próprias, Grocery Store Dataset, GroZi-120 ou Open Images) nesta versão do sistema. A proveniência individual de cada imagem (ficheiro de origem, categoria atribuída, score de clutter e método de seleção) encontra-se documentada de forma rastreável em data/images/manifest.csv, gerado pelo script scripts/build_manifest.py.

III. ANÁLISE VISUAL (COMPONENTE 1 - SHELF_INSPECTOR.PY)

A. Arquitetura de Prompting e Estratégias

O módulo principal de visão computacional assenta sobre o modelo multimodal Gemini 2.5 Flash (configurável através da variável de ambiente GEMINI_MODEL). O sistema efetua a injeção dinâmica do schema estruturado formal definido em prompts/schema.json no marcador {schema} presente nas diretivas textuais. Foram formalizadas três estratégias de engenharia de prompts:

- **Estratégia A (Zero-shot)**: Mapeamento direto de papel, injeção do schema JSON e fornecimento de quatro instruções operacionais concisas, sem a inclusão de exemplos contextuais ou imposição de cadeias de raciocínio.
- **Estratégia B (Chain-of-Thought Visual)**: Determinação de um processo analítico faseado estruturado em quatro etapas obrigatórias que antecedem a emissão do objeto JSON final: (1) Descrição macroscópica da imagem; (2)

Análise segmentada zona a zona; (3) Identificação e taxonomia de anomalias; (4) Classificação e cálculo dos índices métricos.

- **Estratégia C (Few-shot Textual)**: Inclusão de três exemplos de inferência completos no corpo do prompt, contendo descrições e os respetivos objetos JSON idealizados, ilustrando os critérios de severidade e as regras de transição de estados.

B. Avaliação Quantitativa Comparativa

As estratégias foram submetidas a um teste comparativo utilizando um conjunto de validação (ground truth) composto por 15 imagens anotadas manualmente, abrangendo um total de seis tipologias de anomalias distintas. Os testes foram executados com o modelo alternativo gemini-2.5-flash-lite para preservação das quotas principais. Os resultados encontram-se sumarizados na tabela abaixo:

TABLE II
COMPARAÇÃO DAS ESTRATÉGIAS DE PROMPTING (N=15)

Métrica de Desempenho	A (Zero-shot)	B (CoT)	C (Few-shot)
API Success Rate	1.00	1.00	0.93
JSON Parse Rate	0.93	1.00	1.00
Issue Detection Rate (Recall)	0.00	0.17	0.17
False Positive Rate	0.00	0.75	0.89
Severity Accuracy	N/A	1.00	1.00
Hallucination Rate	0.00	0.20	0.30
N.º Tipos de Issue Previstos	0	4	9

C. Análise Crítica e Casos de Falha

- **Estratégia A**: Evidenciou um comportamento excessivamente conservador ("cega mas segura"). Ao classificar a totalidade das amostras como ok e omitir qualquer anomalia, obteve taxas nulas de falsos positivos e alucinações, comprometendo por completo a taxa de desentranhamento (Recall = 0%). Esta abordagem invalida a operação das camadas a jusante (motores de regras e relatórios), uma vez que remove os gatilhos do sistema.
- **Estratégia B**: Demonstrou capacidade de extração semântica ao identificar corretamente o evento empty_shelf na amostra train_106.jpg com precisão métrica de severidade (Severity Accuracy = 1.0). Todavia, induziu falsos positivos em amostras conformes (ex: train_1145.jpg), resultando numa taxa de falsos positivos de 75% e taxa de alucinação de 20%.
- **Estratégia C**: Apesar de manter o mesmo nível de Recall (17%) da Estratégia B, expandiu a variabilidade de predições para nove categorias tipificadas, gerando um aumento substancial de ruído (False Positive Rate = 89%). O prompt extenso aproximou as chamadas dos limites operacionais de Tokens Per Minute (TPM), provocando a emissão de múltiplos erros HTTP 429 pela API e forçando a queda para o estado de fallback na amostra train_111.jpg após a exaustão das cinco tentativas do gestor de backoff exponencial.

Recomendação de Engenharia: A Estratégia B (Chain-of-Thought) foi selecionada como a configuração padrão para o ambiente de produção, por manifestar o equilíbrio mais robusto entre a capacidade de detecção semântica, integridade sintática estrutural e estabilidade de consumo junto da API.

IV. MOTOR DE REGRAS (COMPONENTE 2 — RULE_ENGINE.PY)

A. Pipeline de Compilação JSON e Estrutura de Condições

O motor de regras atua na tradução de especificações operacionais em linguagem natural para esquemas tipificados em JSON. Para este efeito, utiliza o modelo gemini-2.5-flash-lite alimentado pelo prompt parametrizado em prompts/rule_conversion.txt. O mapeamento gera uma estrutura contendo os blocos conditions, action e validation, preenchendo com valores nulos os predicados não declarados.

Através do script scripts/migrate_rules.py, foram geradas e testadas quatro regras de controlo normativo:

TABLE III
REGRAS CONVERTIDAS (DATA/RULES/)

ID	Diretiva em Linguagem Natural	Condições Estruturadas (JSON)	Validação
RULE_001	Alertar se shelf_fill_rate < 0.7 em qualquer zona	fill_rate_threshold: 0.7	is_valid: true
RULE_002	Alertar se houver "muitos" produtos desalinhados	issue_types: ["misaligned"] (sem limiar)	is_valid: false
RULE_003	Alertar se houver wrong_product ≥ severidade média	issue_types: ["wrong_product"], severity_threshold: "medium"	is_valid: true
RULE_004	Alerta crítico se houver empty_shelf com severidade alta	issue_types: ["empty_shelf"], severity_threshold: "high", alert_level: "critical"	is_valid: true

B. Gestão Assíncrona de Ambiguidades

A especificação da regra RULE_002 serve como caso de estudo para o subsistema de tratamento de exceções do compilador. A expressão quantitativa "muitos" carece de correspondência paramétrica direta no schema do sistema (o qual opera com base em thresholds numéricos de área ou severidades discretas).

O parser identificou a inconsistência, definindo o atributo validation.is_valid=false e mapeando o diagnóstico no vetor validation.ambiguities. O sistema armazena a regra no repositório de disco para permitir a sua listagem através da interface de utilizador, mas isola-a de forma determinística das rotinas de avaliação ativa em tempo real através do filtro load_rules(active_only=True), mitigando a ocorrência de falsos positivos operacionais.

C. Execução e Avaliação Semântica

A função evaluate_rule implementa uma semântica estrita do tipo AND para a validação dos predicados declarados. Os filtros contextuais (zone_filter, time_filter) delimitam o escopo espacial e temporal da análise, ao passo que as variáveis funcionais (issue_types, severity_threshold, fill_rate_threshold) determinam as regras de correspondência do conteúdo.

A validação em lote do motor foi executada contra o histórico passivo do sistema através do comando:

```
python src/rule_engine.py --run-all --inspections-dir data/inspections
```

Considerando o universo de 25 registos de inspeção válidos pós-limpeza (avaliando as três diretivas ativas, totalizando 75 avaliações individuais), registou-se um único acionamento positivo: a regra RULE_004 disparou na amostra INS_20260612_090841_023.json devido à presença de um evento empty_shelf de severidade elevada afetando 80% da prateleira na zona Z_S1. Cada evento de processamento é registado de forma incremental no ficheiro de auditoria em formato JSON Lines data/rules/execution_log.jsonl.

V. MEMÓRIA RAG (COMPONENTE 3 — RAG_MEMORY.PY)

A. Estratégias de Chunking e Otimização Local

Para dotar o sistema de capacidades de recuperação histórica sem gerar dependências de rede ou consumo de quotas de chamadas externas, implementou-se uma arquitetura baseada na biblioteca ChromaDB associada ao modelo de embeddings multilingue local paraphrase-multilingual-MiniLM-L12-v2. Foram desenvolvidas e testadas três metodologias distintas de segmentação de dados (chunking) para a indexação dos registos:

- 1) **Mecanismo document:** Cada registo JSON de inspeção é compilado num único bloco condensado de texto construído pela função nativa build_record_text.
- 2) **Mecanismo issue:** O ficheiro é fragmentado em múltiplos blocos independentes correspondentes a cada anomalia individual detetada (ou um bloco nominal padronizado indicando conformidade caso não existam problemas).
- 3) **Mecanismo hybrid:** Executa a indexação concorrente de ambas as estratégias, aplicando um algoritmo corrector de interleaving baseado na ordenação intercalada por relevância dos resultados (round-robin) e subsequente eliminação de duplicados baseada no inspection_id. Esta correção foi introduzida após verificar que os blocos textuais da estratégia document, por possuírem maior densidade e extensão, monopolizavam os primeiros raios de proximidade vetorial.

B. Análise Honesta de Retrieval e Validação de Métricas

A avaliação do acoplamento semântico e da capacidade de recuperação (Recall@3) foi mensurada em dois momentos distintos do ciclo de desenvolvimento, evidenciando o impacto direto da qualidade dos dados indexados:

TABLE IV
RECALL@3 POR ESTRATÉGIA DE CHUNKING

Estratégia	Recall@3 Orig. - 74)	Médio (Corpus Recall@3 Médio Limpo - 25)
document	0.40	0.33
issue (Selecionada)	0.57	0.78
hybrid	0.43	0.33

Análise Crítica dos Resultados: A estratégia focada por anomalia (issue) provou ser consistentemente superior. Com a contração e higienização do corpus para 25 registos válidos, o seu Recall@3 escalou para 78%, uma vez que blocos vetoriais específicos de problemas não sofrem atenuação semântica pelo ruído de resumos globais.

Inversamente, no cenário de corpus reduzido, a estratégia hybrid colapsou, apresentando métricas idênticas à estratégia document (33%). Uma auditoria direta via CLI revelou que o algoritmo de interleaving cessa prematuramente quando os três primeiros vetores vizinhos pertencem todos à classe document devido à baixa variabilidade de distâncias euclidianas num espaço amostral pequeno, evidenciando uma limitação de generalização do modelo híbrido em bases de dados de baixa dimensionalidade.

As rotinas de geração de respostas com síntese de linguagem natural apresentaram pontuações máximas de robustez estrutural (Faithfulness = 1.0; Answer Relevance = 1.0). O sistema comprovou total fidelidade aos dados de origem, abstendo-se de alucinar identificadores de inspeções e ativando os seus mecanismos determinísticos de fallback textual sempre que as chaves de API se encontravam indisponíveis.

C. Diagnóstico e Correção de Contaminação de Dados

Durante a fase de testes e acoplamento de sistemas, detetou-se um comportamento anómalo na base vetorial. O módulo de leitura load_inspection_records() utilizava uma pesquisa por padrão recursiva do tipo glob (*.*.json), o que resultou na indexação indevida de 115 ficheiros temporários gerados em ambiente de desenvolvimento localizados nos subdiretórios _compare/ (108 ficheiros), _scan/ (6 ficheiros) e _test/ (1 ficheiro).

Esta anomalia gerou dois impactos graves na integridade do sistema:

- 1) **Poluição de Domínio:** Injeção de zonas espúrias (Z_CMP, Z_SCAN) no histórico apresentado aos utilizadores.
- 2) **Mascaramento de Registos Históricos:** Um ficheiro residual presente em _compare/ partilhava o mesmo inspection_id de um registo legítimo da zona Z_S1, mas possuía o estado fixado em critical. Devido à ordenação ASCII na construção do dicionário de dados ("_" superior a "I"), o artefacto de teste sobrepunha silenciosamente a entrada real da base de dados, induzindo o sistema a reportar falsos alarmes críticos para a zona Z_S1.

Ação Corretiva: Os 115 ficheiros de desenvolvimento foram segregados e movidos para o diretório isolado data/dev artifacts/. Procedeu-se à purga e reindexação completa da vectorstore, restabelecendo o universo estrito de 25 registos operacionais válidos.

VI. GERADOR DE RELATÓRIOS (COMPONENTE 4 — REPORT_GENERATOR.PY)

A. Integração e Conceito de Sessão Operacional

O componente de relatórios consolida os outputs gerados pelos três módulos precedentes, estruturando um documento final em sintaxe Markdown composto por cinco secções normativas: Sumário Executivo, Problemas por Zona, Regras Disparadas, Contexto Histórico Relevante e Recomendações.

O motor de geração adota o conceito de Sessão, processando matrizes de N ficheiros de inspeção de forma agregada para contextualizar o estado global da operação. Para assegurar a eficiência de custos e mitigar dependências externas, o documento é gerado de forma estritamente determinística através do processamento de templates locais, sem efetuar chamadas a modelos de linguagem.

Nota sobre a Integração com o Projeto 1: a integração com a componente de afluência de loja do Projeto 1, referida na Secção 6.6/13 do enunciado, tem carácter opcional e não foi implementada nesta versão do sistema — ver Secção IX (Trabalho Futuro).

B. Output do Relatório de Sessão Completa (Pós-Higienização)

Abaixo apresenta-se o documento final gerado de forma programática pelo sistema após a remoção dos artefactos de desenvolvimento e inclusão do novo ponto de controlo de gôndola (Z_S3) gerado durante os testes de integração do sistema. O conteúdo integral encontra-se no Anexo D; reproduz-se aqui o Sumário Executivo, as Regras Disparadas, o Contexto Histórico Relevante e as Recomendações:

```

Relatorio de Sessao
Gerado em: 2026-06-12T11:34:37Z
Inspecoes incluidas: INS_20260610_183237_001,
INS_20260610_183302_001, INS_20260612_090234_001,
INS_20260612_090236_002, ..., INS_20260612_093936_072

1. Sumario Executivo
Nesta sessao foram analisadas 75 inspecoes abrangendo
3 zonas operacionais. Estado geral do parque de
prateleiras: 0 em estado critico, 12 em estado de
atencao e 13 em conformidade (sem problemas
detetados). No computo geral, foram identificadas 33
anomalias ativas distribuidas pelas gondolas
monitorizadas. Adicionalmente, registou-se que 50
inspecoes nao produziram dados validos devido a
falhas temporarias na comunicacao com a API de visao
computacional, encontrando-se listadas de forma
segregada na Seccao 2 para efeitos de reproprocessamento
tecnico.

3. Regras Disparadas
- RULE_004 [Nivel: critical] verificado em
INS_20260612_090841_023 (Zona: Z_S1, Data:
2026-06-12): Alerta CRITICO: prateleira vazia com
severidade alta detetada na zona Z_S1.

4. Contexto Historico Relevante
- Sem historico relevante para os problemas detetados
nesta sessao.

```

(Nota de Sistema: Atendendo a que o escopo de análise da presente sessão engloba a totalidade do universo indexado pos-higienização da base vetorial, a pesquisa de vizinhos próximos externos resulta, de forma logicamente correta, num conjunto vazio).

5. Recomendacoes (Ordenadas por Criticidade)
1. [RULE_004] Emitir ordem de intervencao operacional imediata para a zona Z_S1 (Rutura critica de stock com 80% de area afetada detetada na prateleira inferior).
 2. [Reposicao] Agendar reabastecimento de urgencia para a gondola Z_S2 devido a deteccao de areas vagas.
 3. [Reposicao] Alocar equipa de reposicao para mitigar os pontos de rutura parciais registados na zona Z_S1.
 4. [Arrumacao] Executar plano de alinhamento e facing de produtos na zona Z_S1 para retificacao de itens deslocados.
 5. [Auditoria] Realizar verificacao manual preventiva dos desvios operacionais assinalados como "Outro problema" na zona Z_S1.

A listagem zona a zona, inspeção a inspeção, encontra-se reproduzida na íntegra no Anexo D (Secção 2 do relatório de sessão).

VII. INTERFACE CLI (COMPONENTE 5 — INTERFACE.PY)

A. Arquitetura de Comandos e Correção Estrutural

O script `src/interface.py` atua como a camada de abstração e orquestração unificada, expondo as capacidades operacionais do sistema através de subcomandos estruturados na linha de comandos:

```
python src/interface.py [inspect | add rule | list rules |
delete rule | test rule | history | compare | report]
```

Resolução de Bug Bloqueante: Na fase de acoplamento do sistema, identificou-se um erro sintático fatal (Indentation-Error: expected an indented block after class definition on line 32). A declaração da exceção customizada `CLIErrror` tinha sido inadvertidamente fundida com um comentário de secção na mesma linha (class `CLIErrror(Exception): #estado de sessao`), deixando a classe sem um bloco de corpo válido. O código foi corrigido aplicando uma docstring normativa:

```
class CLIErrror(Exception):
    """Erro amigavel apresentado ao utilizador sem
    stack trace."""
```

B. Verificação de Integração E2E dos Subcomandos

Após a resolução do erro sintático, validou-se a árvore de comandos integrando os módulos do sistema:

- **Subcomando `list rules`:** Demonstrou acoplamento correto com o Motor de Regras, expondo as diretivas e listando a `RULE_002` com a etiqueta flag [com ambiguidades], garantindo a transparência ao operador enquanto assegura a sua exclusão das rotinas automáticas de varredura.
- **Subcomando `history`:** A execução do comando `python src/interface.py history "problemas na zona Z_S1" -n 3` recuperou as fontes estritamente pertencentes ao espaço amostral real purgado, eliminando as antigas referências cruzadas que apontavam para as zonas de teste fictícias.

- **Subcomando `compare`:** O comando realiza a consolidação estatística comparativa entre matrizes de zonas. A tabela abaixo ilustra o comportamento da CLI ao comparar as zonas reais `Z_S1` e `Z_S2`:

TABLE V
PYTHON SRC/INTERFACE.PY COMPARE Z_S1 Z_S2

Métrica Consolidada	Z_S1	Z_S2
Número de Inspeções	23	1
Preenchimento Médio	95%	92%
Inspeções em Estado 'OK'	12	0
Inspeções em Estado 'Atenção'	11	1
Inspeções em Estado 'Crítico'	0	0
Anomalias: Prateleira Vazia	10	1
Anomalias: Produtos Desalinhados	16	0
Anomalias: Outro Problema	6	0

Validação de Integridade de Dados: Antes da purga efetuada na base vetorial, o subcomando `compare` reportava falsamente um total de 24 inspeções e uma instância em estado Crítico para a zona `Z_S1`. Esta anomalia derivava da colisão de IDs com o ficheiro de teste oculto em `_compare/`, cuja resolução restabeleceu os valores legítimos (23 inspeções, zero críticos).

VIII. HARNESS DE AVALIAÇÃO COLETIVA (EVALUATE.PY)

A. Estrutura Metrológica do Harness

O módulo `evaluate.py` centraliza a extração e consolidação de métricas operacionais do sistema, gerando o relatório unificado `data/evaluation/evaluation_report.json`. O componente atua como agregador analítico das fases de validação, aferindo a conformidade sintática e semântica de cada módulo face às especificações originais.

B. Consolidação de Resultados Estruturais

- **Métricas de Visão (Componente 1):** Reatribuição dos índices obtidos na Fase 2 através da leitura direta de `strategy_comparison.json`, confirmando o teto de Recall em 17% para chaves baseadas em CoT e poupança de chamadas redundantes.
- **Métricas Normativas (Componente 2):** Registou eficácia máxima nos três indicadores de integridade estrutural do compilador de regras (Rule Parse Rate = 1.0; Rule Correctness = 1.0; Ambiguity Detection Rate = 1.0) sobre o universo de inspeções válidas pós-limpeza, comprovando a aderência ao esquema e o correto isolamento de ambiguidades.
- **Métricas de Recuperação Semântica (Componente 3):** Consolidação dos índices de Recall da base vetorial (evidenciando a superioridade da divisão por issue com 78% de acerto) e validação dos coeficientes de qualidade de resposta (Faithfulness = 1.0; Answer Relevance = 1.0).

Resolução de Erro de Percurso de Diretório: Identificou-se que a rotina `find_cached_record()` no harness quebrava a execução ao tentar acionar o avaliador secundário LLM-as-judge via parâmetro `-llm-judge`. O algoritmo apontava de

forma estática para a constante desatualizada COMPARE_DIR = data/inspections/_compare/, diretório que tinha sido purgado no processo de higienização de dados. Corrigiu-se o apontador para o novo endereço de arquivo data/dev_artifacts/_compare/, reativando a execução do avaliador baseado no modelo gemini-3-flash-preview (o qual atribuiu uma pontuação média de 4.33/5 às análises da Estratégia B, reportando apenas pequenas truncagens na extensão do campo descritivo model_reasoning).

IX. CONCLUSÕES E TRABALHO FUTURO

A. Diagnóstico de Sucessos Operacionais

- **Pipeline Funcional Ponta-a-Ponta:** O sistema cumpre com sucesso o fluxo completo de processamento (ingestão → análise visual multimodal → inferência lógica de regras → indexação e pesquisa semântica → compilação de relatórios estruturados → CLI).
- **Resiliência a Falhas:** Os componentes de tratamento de exceções e fallbacks graciosos demonstraram elevada robustez em cenários de stress real e esgotamento de limites das APIs, mantendo a integridade sintática e operacional dos relatórios gerados.
- **Qualidade do Isolamento Normativo:** O motor de regras demonstrou capacidade de triagem de ambiguidades textuais, isolando diretivas malformadas sem bloquear a execução das regras validadas.
- **Eficiência de Custos:** A indexação vetorial e os mecanismos de cache baseados em criptografia MD5 local demonstraram eficácia na redução de custos operacionais e na preservação de quotas computacionais.

B. Diagnóstico de Limitações Identificadas

- **Baixo Desempenho do Modelo de Visão:** O nível macro de recall visual obtido (17%) revela-se insuficiente para operações de escala industrial. O modelo gemini-2.5-flash-lite configurado com temperatura nula tendeu a assumir posições excessivamente conservadoras, omitindo anomalias de relevância operacional.
- **Sensibilidade do Chunking Híbrido:** A estratégia híbrida de recuperação provou ser instável e sensível à escala volumétrica do corpus indexado, perdendo capacidade de diferenciação analítica em bases de dados reduzidas.
- **Vulnerabilidade na Gestão de Ficheiros:** A ausência de isolamento inicial a nível de sistema de ficheiros permitiu a infiltração de artefactos de teste no pipeline de produção, gerando impactos ocultos em múltiplos componentes antes da sua deteção na fase de integração final.

C. Linhas de Desenvolvimento Futuro

Para futuras iterações do sistema, recomendam-se as seguintes intervenções técnicas:

- 1) **Isolamento Estrito de Ambientes:** Segregação nativa a nível de configuração de software entre os diretórios de persistência de dados reais (data/inspections/) e os repositórios de testes ou desenvolvimento (data/dev_artifacts/).

- 2) **Upgrade do Modelo de Visão:** Substituição do modelo computacional leve pelo ecossistema completo gemini-2.5-flash e calibração fina do prompt CoT para incrementar a sensibilidade na deteção de anomalias subtis, fixando thresholds aceitáveis de falsos positivos.
- 3) **Expansão da Indexação Semântica:** Alargar o âmbito de atuação do RAG através da indexação ativa do ficheiro de logs do motor de regras (execution_log.jsonl), capacitando o comando history a responder com exatidão a interrogações relativas à frequência de alertas e regras disparadas.
- 4) **Controlo Volumétrico de Tokens:** Integração de um gestor métrico de volume de tokens por unidade de tempo no módulo QuotaManager, expandindo a atual limitação focada apenas em contagens brutas de requisições por minuto (RPM).
- 5) **Diálogo de Clarificação de Ambiguidades:** Implementar o diálogo interativo previsto na Secção 5.4 do enunciado: atualmente, uma regra ambígua (RULE_002) é guardada e marcada como inválida, mas o sistema não pergunta interativamente ao gestor como resolver cada ambiguidade antes de a persistir.
- 6) **Integração com o Projeto 1:** funcionalidade opcional não implementada nesta versão do sistema (Secção 6.6/13 do enunciado). Ficaria a cargo desta integração enriquecer a vectorstore do RAG com contexto de afluência (dados de trajetória do Projeto 1), permitindo distinguir, por exemplo, se uma prateleira vazia se deve a procura elevada ou a falha de reposição.

APPENDIX

Listados na íntegra em prompts/.

A. schema.json — schema JSON do inspection_record

```
{
  "inspection_id": "INS_YYYYMMDD_HHMMSS_NNN",
  "timestamp": "YYYY-MM-DDTHH:MM:SSZ",
  "image_path": "path/to/image.jpg",
  "zone_id": "Z_S3",
  "overall_status": "ok|warning|critical",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "empty_shelf|wrong_product|damaged|misaligned|label_missing|other",
      "location": "ex: prateleira inferior, lado esquerdo",
      "severity": "low|medium|high",
      "description": "descricao em linguagem natural do problema",
      "confidence": 0.0,
      "affected_area_pct": 0.0
    }
  ],
  "shelf_fill_rate": 0.0,
  "products_detected": ["lista de categorias de produto visíveis"],
  "model_reasoning": "cadeia de raciocinio explicita antes da classificacao"
}
```

B. strategy_a_zero_shot.txt

Define o papel do modelo + schema + 4 instruções diretas, sem exemplos.

Es um sistema de inspecao automatica de prateleiras de retalho.

Analisa a imagem de prateleira fornecida e produz uma analise estruturada em JSON, seguindo EXATAMENTE o schema abaixo (nao acrescentes nem removas campos):

```
{schema}
```

Instrucoes:

- "overall_status" deve ser "ok", "warning" ou "critical".
- "issues" e uma lista (pode ser vazia) de problemas detetados: prateleiras vazias, produtos errados, danificados, desalinhados, etiquetas em falta, etc.
- "shelf_fill_rate" e um valor entre 0.0 e 1.0 que estima a percentagem de espaco de prateleira ocupado por produtos.
- "products_detected" e uma lista de categorias de produtos visiveis (em portuges, ex: "cafe", "detergente", "conservas").
- "model_reasoning" deve conter uma breve explicacao do raciocinio usado para chegar a classificacao.

Responde APENAS com o objeto JSON, sem texto adicional, sem blocos de codigo markdown.

C. strategy_b_cot.txt

Pede raciocínio estruturado em 4 passos (DESCRIÇÃO GERAL → ANÁLISE ZONA A ZONA → IDENTIFICAÇÃO DE ANOMALIAS → CLASSIFICAÇÃO FINAL) antes do JSON final. Estratégia recomendada (Secção III).

Es um sistema de inspecao automatica de prateleiras de retalho. Tens de raciocinar passo a passo, de forma explicita, ANTES de produzir a classificacao final.

Segue rigorosamente este processo de raciocinio:

1. DESCRICAO GERAL: descreve o que ves na imagem (tipo de prateleira, numero de niveis/prateleiras visiveis, tipos de produtos predominantes).
2. ANALISE ZONA A ZONA: percorre a imagem por secoes (prateleira superior, intermedia, inferior; esquerda, centro, direita) e descreve o estado de cada zona - cheia, parcialmente vazia, vazia, desorganizada, com produtos tombados/danificados/etiquetas em falta, etc.
3. IDENTIFICACAO DE ANOMALIAS: para cada zona problematica identificada no passo 2, classifica a anomalia (empty_shelf, wrong_product, damaged, misaligned, label_missing, other), estima a severidade (low/medium/high) e a percentagem de area afetada.
4. CLASSIFICACAO FINAL: com base nos passos anteriores, determina o overall_status ("ok" se nao ha problemas relevantes, "warning" se ha problemas menores, "critical" se ha problemas graves como prateleiras totalmente vazias ou multiplos problemas de severidade alta), e estima o shelf_fill_rate global (0.0 a 1.0).

Depois de completares este raciocinio internamente, produz APENAS um objeto JSON seguindo EXATAMENTE o schema abaixo (nao acrescentes nem removas campos). O campo "model_reasoning" deve conter o resumo do raciocinio dos passos 1-4 acima, de forma explicita e verificavel face a imagem.

```
{schema}
```

Responde APENAS com o objeto JSON, sem texto adicional, sem blocos de codigo markdown.

D. strategy_c_fewshot.txt

3 exemplos completos (descrição + JSON correto) antes do pedido.

Es um sistema de inspecao automatica de prateleiras de retalho. Abaixo estao exemplos de analises corretas feitas anteriormente sobre outras imagens (descritas em texto, ja que nao podemos reenviar as imagens originais). Usa-os como referencia do nivel de detalhe e do formato esperado.

--- EXEMPLO 1 ---

Imagem: prateleira de bebidas, 4 niveis, bem preenchida, sem produtos fora de posicao. Pequena zona vazia no canto superior direito (~5% da area).

Analise correta:

```
{
  "overall_status": "ok",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "empty_shelf",
      "location": "prateleira superior, lado direito",
      "severity": "low",
      "description": "Pequeno espaco vazio no canto superior direito, provavelmente produto vendido recentemente.",
      "confidence": 0.7,
      "affected_area_pct": 0.05
    }
  ],
  "shelf_fill_rate": 0.93,
  "products_detected": ["refrigerantes", "agua engarrafada", "sumos"],
  "model_reasoning": "A prateleira esta quase completamente cheia e organizada. Apenas o canto superior direito apresenta um pequeno espaco vazio, sem outros sinais de desorganizacao ou produtos fora de posicao. Por isso o estado geral e 'ok' com um issue de baixa severidade."
}
```

--- EXEMPLO 2 ---

Imagem: prateleira de produtos de limpeza, nivel inferior completamente vazio (sem produtos nem etiquetas visiveis), niveis superiores normais.

Analise correta:

```
{
  "overall_status": "critical",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "empty_shelf",
      "location": "prateleira inferior, largura total",
      "severity": "high",
      "description": "Prateleira inferior completamente vazia, sem produtos nem etiquetas de preco visiveis, sugerindo rutura de stock prolongada.",
      "confidence": 0.95,
      "affected_area_pct": 0.25
    }
  ],
  "shelf_fill_rate": 0.7,
  "products_detected": ["detergente liquido", "amaciador", "lixivia"],
  "model_reasoning": "Os niveis superiores estao bem preenchidos e organizados. O nivel inferior, no entanto, esta totalmente vazio numa largura significativa, o que reduz drasticamente o fill rate global e justifica severidade alta e estado critical."
}
```



```
}

--- EXEMPLO 3 ---
Imagem: prateleira de conservas, maioria organizada, mas uma
lata esta tombada
de lado no nivel intermedio e duas embalagens estao trocadas
de categoria.
Analise correta:
{
  "overall_status": "warning",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "damaged",
      "location": "prateleira intermedia, secao central",
      "severity": "medium",
      "description": "Lata de conserva tombada de lado entre
        as restantes embalagens verticais.",
      "confidence": 0.85,
      "affected_area_pct": 0.03
    },
    {
      "issue_id": "ISS_002",
      "type": "wrong_product",
      "location": "prateleira intermedia, lado esquerdo",
      "severity": "low",
      "description": "Duas embalagens de produto de categoria
        diferente misturadas com as conservas.",
      "confidence": 0.6,
      "affected_area_pct": 0.02
    }
  ],
  "shelf_fill_rate": 0.9,
  "products_detected": ["conservas de atum", "conservas de
    feijao", "conservas de milho"],
  "model_reasoning": "A prateleira esta bem preenchida no
    geral, mas foram identificados dois problemas
    pontuais: uma lata tombada (severidade media, indica
    falta de manutencao) e produtos fora da categoria
    correta (severidade baixa, possivel erro de reposicao
    ). Nao ha problemas suficientemente graves para
    'critical', mas merece atencao, logo 'warning'."
}

--- FIM DOS EXEMPLOS ---

Agora analisa a NOVA imagem fornecida. Produz APENAS um
objeto JSON seguindo
EXATAMENTE o schema abaixo (nao acrescentes nem removas
campos). O campo
"model_reasoning" deve explicar o raciocinio que levou a
classificacao,
segundo o mesmo nivel de detalhe dos exemplos.

{schema}

Responde APENAS com o objeto JSON, sem texto adicional, sem
blocos de codigo
markdown.
```

E. rule_conversion.txt

Converte regra em linguagem natural → JSON estruturado
(conditions/action/validation), usado pelo rule_engine.py.

```
Es um sistema que converte regras de deteccao de anomalias
em prateleiras de
retalho, descritas em linguagem natural por um gestor de
loja, para um
formato JSON estruturado que pode ser executado
automaticamente sobre
"inspection records" gerados por um sistema de visao
computacional.

Cada inspection record tem os seguintes campos relevantes:
- "timestamp": data/hora ISO 8601 UTC da inspecao
- "overall_status": "ok" | "warning" | "critical"
- "shelf_fill_rate": numero entre 0.0 e 1.0 (percentagem de
  preenchimento)
- "zone_id": identificador da zona/corredor (ex: "Z_S1")
- "issues": lista de objetos com "type" (empty_shelf |
  wrong_product | damaged
```

```
| misaligned | label_missing | other), "severity" (low |
  medium | high),
"location" (descricao livre, ex: "prateleira inferior,
lado esquerdo") e
"affected_area_pct" (0.0 a 1.0)

A tua tarefa: ler a descricao em linguagem natural fornecida
pelo utilizador
e produzir APENAS um objeto JSON seguindo EXATAMENTE o
schema abaixo (nao
acrescentes nem removas campos).

Regras importantes:
- "natural_language": copia exatamente o texto original
  fornecido pelo
  utilizador.
- "description": reformula a regra de forma clara, completa
  e inequivoca, em
  portugues formal, tal como seria escrita num documento de
  especificacao.
- "conditions": preenche apenas os campos relevantes para a
  regra; os
  restantes devem ficar a 'null'.
- "zone_filter": lista de zone_id se a descricao mencionar
  zona(s)
  especifica(s); 'null' se se aplicar a todas as zonas.
- "time_filter": objeto {"hours_start": int, "hours_end":
  int} (0-23, UTC)
  se a descricao mencionar um intervalo horario; 'null'
  caso contrario.
- "issue_types": lista de tipos de issue relevantes; 'null'
  se a regra nao
  depender de issues especificos (ex: depende apenas de
  shelf_fill_rate).
- "severity_threshold": "low" | "medium" | "high" se a
  descricao
  mencionar um nivel minimo de severidade; 'null' caso
  contrario.
- "fill_rate_threshold": numero 0.0-1.0 se a descricao
  mencionar um limiar
  de preenchimento; 'null' caso contrario.
- "location_filter": "bottom" | "middle" | "top" | "any"
  se a descricao
  mencionar uma zona da prateleira (inferior/meio/superior)
  ; 'null' caso
  contrario.
- "action.alert_level": escolhe "info", "warning" ou "
  critical" de acordo
  com a urgencia implicita na descricao (default razoavel: "
  warning").
- "action.notification_message": mensagem curta e clara para
  o gestor de
  loja, podendo usar os placeholders {zone_id}, {issue_type
  }, {severity} e
  {shelf_fill_rate}.
- "validation.is_valid": 'false' se a descricao for ambigua,
  vaga ou faltar
  informacao essencial (ex: nao especifica um limiar
  numerico onde seria
  necessario, ou usa termos sem correspondencia clara nos
  campos
  disponiveis); 'true' caso contrario.
- "validation.ambiguities": lista de strings descrevendo os
  aspetos
  ambiguos identificados (lista vazia se "is_valid" for '
  true').
- "validation.assumptions": lista de strings descrevendo
  qualquer suposicao
  feita para preencher campos nao especificados
  explicitamente (mesmo que a
  regra seja considerada valida).

Descricao em linguagem natural fornecida pelo utilizador:
"""
{nl_description}
"""

Schema (preenche EXATAMENTE estes campos, sem adicionar "
rule_id" nem
"created_at" - esses sao gerados pelo sistema):
{schema}

Responde APENAS com o objeto JSON, sem texto adicional, sem
```

blocos de código
markdown.

F. llm_judge.txt

Usado pelo `-llm-judge` do `evaluate.py` para comparar um `inspection_record` gerado com o `ground truth` e atribuir uma pontuação 1-5.

Es um avaliador independente (LLM-as-judge) de um sistema de visão computacional para inspeção de prateleiras de retalho.

Tens acesso a:

1. Uma referência (`ground truth`) escrita por um humano após inspeção visual real da imagem.
2. A saída produzida automaticamente pelo sistema (estado geral, problemas detetados e raciocínio do modelo).

Avalia a QUALIDADE da saída do sistema face a referência, numa escala de 1 a 5, considerando tanto a classificação (`overall_status`, `issues`) como a qualidade e correção da análise textual:

1 = completamente incorreta ou irrelevante face a referência
2 = maioritariamente incorreta, com erros significativos
3 = parcialmente correta, com omissões ou erros irrelevantes
4 = maioritariamente correta, com pequenas imprecisões
5 = totalmente correta e consistente com a referência

Referência (`ground truth`):

```
"""
{ground_truth}
"""
```

Saída do sistema:

```
"""
{system_output}
"""
```

Responde APENAS com um objeto JSON no formato exato:
{`"score"`: <número inteiro de 1 a 5>, `"justification"`: "< frase curta em português>"}
Sem texto adicional, sem blocos de código markdown.

G. B.1 — Estado ok (sem problemas)

```
{
  "inspection_id": "INS_20260612_090234_001",
  "timestamp": "2026-06-12T09:02:34Z",
  "image_path": "data/images/normal/train_1066.jpg",
  "zone_id": "Z_S1",
  "overall_status": "ok",
  "issues": [],
  "shelf_fill_rate": 0.98,
  "products_detected": [
    "Dove body wash", "Olay body wash", "Nivea body wash",
    "Aveeno body wash", "Dove bar soap", "Camay bar soap"
  ],
  "model_reasoning": "1. DESCRICAO GERAL: ... prateleira bem abastecida, organizada... 4. CLASSIFICACAO FINAL: Nao ha problemas relevantes que justifiquem 'warning' ou 'critical'.",
  "_strategy": "B"
}
```

H. B.2 — Estado warning com múltiplas issues

```
{
  "inspection_id": "INS_20260612_090310_004",
  "timestamp": "2026-06-12T09:03:10Z",
  "image_path": "data/images/normal/train_1138.jpg",
  "zone_id": "Z_S1",
  "overall_status": "warning",
  "issues": [
```

```
  {"issue_id": "ISS_001", "type": "misaligned", "location": "prateleira 3, centro-direita", "severity": "low", "description": "Algumas garrafas de gel de banho Dial estao ligeiramente desalinhadas"},
  {"issue_id": "ISS_003", "type": "empty_shelf", "location": "prateleira 4, centro", "severity": "low", "description": "Pequena lacuna na seccao de sabonetes Olay.", "confidence": 1.0, "affected_area_pct": 1.0},
  {"issue_id": "ISS_006", "type": "other", "location": "prateleira 5, varios pontos", "severity": "low", "description": "Algumas etiquetas de preco estao ligeiramente tortas.", "confidence": 1.0, "affected_area_pct": 1.0}
],
"shelf_fill_rate": 0.95,
"products_detected": ["Olay Body Wash", "Dove Bar Soap", "...", "Coast Bar Soap"],
"model_reasoning": "1. DESCRICAO GERAL: ... 4. CLASSIFICACAO FINAL: O 'overall_status' e 'warning' devido a presenca de multiplas anomalias menores de desalinhamento e pequenas lacunas.",
"_strategy": "B"
}
```

(6 issues no total nesta inspeção — ver lista completa na Secção 2 do Anexo D, zona Z_S1)

I. B.3 — Estado warning com alerta crítico associado (RULE_004)

```
{
  "inspection_id": "INS_20260612_090841_023",
  "timestamp": "2026-06-12T09:08:41Z",
  "image_path": "data/images/normal/train_2057.jpg",
  "zone_id": "Z_S1",
  "overall_status": "warning",
  "issues": [
    {"issue_id": "ISS_001", "type": "empty_shelf", "location": "prateleira inferior, lado esquerdo", "severity": "high", "description": "Uma porcao significativa da prateleira inferior esta vazia, com apenas alguns produtos de sabonete Coast visiveis.", "confidence": 0.95, "affected_area_pct": 80.0},
    {"issue_id": "ISS_002", "type": "misaligned", "location": "prateleira 4, lado esquerdo", "severity": "low", "description": "Algumas caixas de sabonete Irish Spring estao ligeiramente desalinhadas.", "confidence": 0.85, "affected_area_pct": 10.0},
    {"issue_id": "ISS_003", "type": "other", "location": "chao, perto da prateleira inferior esquerda", "severity": "low", "description": "Uma etiqueta de preco esta visivel no chao.", "confidence": 0.9, "affected_area_pct": 0.0}
  ],
  "shelf_fill_rate": 0.95,
  "products_detected": ["body wash", "shampoo", "bar soap", "bath salts", "children's bath products"],
  "model_reasoning": "... 4. CLASSIFICACAO FINAL: o 'overall_status' e 'warning' devido a seccao significativamente vazia na prateleira inferior (severidade alta, 80% da area).",
  "_strategy": "B"
}
```

Nota: o `issue_id=ISS_001` (`empty_shelf`, severidade high) é o que dispara `RULE_004`, gerando o único alerta crítico de toda a sessão — ver Secção IV e Anexo D, Secção 3.

J. B.4 — Registo fallback (falha de API)

```
{
  "inspection_id": "INS_20260612_090236_002",
  "timestamp": "2026-06-12T09:02:36Z",
  "image_path": "data/images/normal/train_1098.jpg",
```

```

"zone_id": "Z_S1",
"overall_status": "unknown",
"issues": [],
"shelf_fill_rate": null,
"products_detected": [],
"model_reasoning": "FALLBACK: nao foi possivel analisar
esta imagem. Motivo: Erro na chamada a API: 503
UNAVAILABLE. ... 'This model is currently
experiencing high demand...'",
"_fallback": true,
"_error": "Erro na chamada a API: 503 UNAVAILABLE. ..."
}

```

Registos `_fallback=true` são incluídos no relatório de sessão (Secção 2, listados em separado por zona) mas excluídos de `load_inspection_records()` — não entram no corpus RAG nem na avaliação de regras (Secções V e IV).

K. C.1 — RULE_001 (válida, sem ambiguidades)

```

{
  "rule_id": "RULE_001",
  "natural_language": "Alertar sempre que o shelf_fill_rate
de uma prateleira for inferior a 0.7",
  "description": "O sistema deve emitir um alerta sempre que
a taxa de preenchimento (shelf_fill_rate) de
qualquer zona for inferior a 0.7 (70%).",
  "conditions": {
    "zone_filter": null, "time_filter": null, "issue_types":
null,
    "severity_threshold": null, "fill_rate_threshold": 0.7, "
location_filter": null
  },
  "action": {
    "alert_level": "warning",
    "notification_message": "Alerta: a zona {zone_id} tem uma
taxa de preenchimento de {shelf_fill_rate}, abaixo
do limiar de 70%."
  },
  "validation": {
    "is_valid": true, "ambiguities": [],
    "assumptions": ["Aplica-se a todas as zonas, dado que a
descricao nao especifica nenhuma zona em particular
."]
  }
}

```

L. C.2 — RULE_002 (conversão ambígua — não ativada)

```

{
  "rule_id": "RULE_002",
  "natural_language": "Alertar quando houver muitos produtos
fora de posicao numa prateleira",
  "description": "O sistema deve emitir um alerta quando
forem detetados produtos desalinhados (misaligned)
numa prateleira, em quantidade considerada elevada.",
  "conditions": {
    "zone_filter": null, "time_filter": null,
    "issue_types": ["misaligned"], "severity_threshold": "
medium",
    "fill_rate_threshold": null, "location_filter": null
  },
  "action": {
    "alert_level": "warning",
    "notification_message": "Alerta: foram detetados produtos
desalinhados na zona {zone_id} (severidade {
severity})."
  },
  "validation": {
    "is_valid": false,
    "ambiguities": [
      "A descricao nao especifica o que significa 'muitos'
produtos fora de posicao (ex: percentagem de area
afetada ou numero de issues), pelo que nao e
possivel definir um limiar fiavel."
    ]
  },
  "assumptions": [

```

```

    "Assumiu-se severidade minima 'medium' como proxy
provisorio para 'muitos', mas o schema de
condicoes nao permite expressar um limiar de area
afetada."
  ]
}

```

Esta regra fica gravada e visível (list rules mostra [com ambiguidades]), mas é excluída de `load_rules(active_only=True)` — exemplo direto da estratégia de deteção de ambiguidades (Secção IV).

M. C.3 — RULE_003 (válida, com aproximação documentada)

```

{
  "rule_id": "RULE_003",
  "natural_language": "Alertar quando houver muitos produtos
fora de posicao numa prateleira (mais de 20% da area
, severidade media ou superior)",
  "description": "O sistema deve emitir um alerta quando for
detetado pelo menos um issue do tipo 'produto fora
de posicao' (wrong_product) com severidade media ou
superior em qualquer zona.",
  "conditions": {
    "zone_filter": null, "time_filter": null,
    "issue_types": ["wrong_product"], "severity_threshold": "
medium",
    "fill_rate_threshold": null, "location_filter": null
  },
  "action": {
    "alert_level": "warning",
    "notification_message": "Alerta: produtos fora de posicao
detetados na zona {zone_id} com severidade {
severity}."
  },
  "validation": {
    "is_valid": true, "ambiguities": [],
    "assumptions": [
      "O limiar original de 'mais de 20% da area afetada' nao
e representavel no schema de condicoes definido (
que nao inclui affected_area_pct); usa-se apenas o
filtro de severidade >= 'medium' como aproximacao
."
    ]
  }
}

```

N. C.4 — RULE_004 (válida, crítica — única regra dispareada)

```

{
  "rule_id": "RULE_004",
  "natural_language": "Marcar como critico qualquer
prateleira que tenha um problema do tipo prateleira
vazia (empty_shelf) com severidade alta",
  "description": "O sistema deve emitir um alerta critico
sempre que for detetado um issue do tipo 'prateleira
vazia' (empty_shelf) com severidade alta em qualquer
zona.",
  "conditions": {
    "zone_filter": null, "time_filter": null,
    "issue_types": ["empty_shelf"], "severity_threshold": "
high",
    "fill_rate_threshold": null, "location_filter": null
  },
  "action": {
    "alert_level": "critical",
    "notification_message": "Alerta CRITICO: prateleira vazia
com severidade alta detetada na zona {zone_id}."
  },
  "validation": {
    "is_valid": true, "ambiguities": [], "assumptions": []
  }
}

```

Relatório de Sessão
Gerado em: 2026-06-12T11:53:11Z
*Inspecoes incluidas: INS_20260610_183237_001,
INS_20260610_183302_001, INS_20260612_090234_001,
INS_20260612_090236_002, INS_20260612_090254_003,
INS_20260612_090310_004, INS_20260612_090324_005,
INS_20260612_090340_006, INS_20260612_090359_007,
INS_20260612_090422_008, INS_20260612_090442_009,
INS_20260612_090458_010, INS_20260612_090517_011,
INS_20260612_090538_012, INS_20260612_090557_013,
INS_20260612_090612_014, INS_20260612_090627_015,
INS_20260612_090647_016, INS_20260612_090701_017,
INS_20260612_090714_018, INS_20260612_090716_019,
INS_20260612_090729_020, INS_20260612_090740_021,
INS_20260612_090810_022, INS_20260612_090841_023,
INS_20260612_090853_024, INS_20260612_090935_025,
INS_20260612_091030_026, INS_20260612_091135_027,
INS_20260612_091216_028, INS_20260612_091255_029,
INS_20260612_091343_030, INS_20260612_091417_031,
INS_20260612_091455_032, INS_20260612_091539_033,
INS_20260612_091621_034, INS_20260612_091719_035,
INS_20260612_091817_036, INS_20260612_091857_037,
INS_20260612_091939_038, INS_20260612_092011_039,
INS_20260612_092048_040, INS_20260612_092120_041,
INS_20260612_092152_042, INS_20260612_092225_043,
INS_20260612_092257_044, INS_20260612_092330_045,
INS_20260612_092403_046, INS_20260612_092441_047,
INS_20260612_092528_048, INS_20260612_092601_049,
INS_20260612_092636_050, INS_20260612_092708_051,
INS_20260612_092741_052, INS_20260612_092813_053,
INS_20260612_092847_054, INS_20260612_092919_055,
INS_20260612_092952_056, INS_20260612_093025_057,
INS_20260612_093058_058, INS_20260612_093130_059,
INS_20260612_093207_060, INS_20260612_093252_061,
INS_20260612_093329_062, INS_20260612_093406_063,
INS_20260612_093440_064, INS_20260612_093515_065,
INS_20260612_093548_066, INS_20260612_093625_067,
INS_20260612_093705_068, INS_20260612_093745_069,
INS_20260612_093819_070, INS_20260612_093854_071,
INS_20260612_093936_072, INS_20260612_113925_001*

1. Sumario Executivo

Nesta sessão foram analisadas 75 inspecao(oes) abrangendo 3 zona(s). Estado geral: 0 em estado critico, 12 em atencao e 13 sem problemas . No total foram detetados 33 problema(s) entre todas as inspecoes. Adicionalmente, 50 inspecao(oes) nao produziram dados validos (falha temporaria na chamada a API) e estao listadas em separado na Seccao 2.

2. Problemas por Zona

Zona Z-S1

- Inspecao 'INS_20260610_183237_001' (2026-06-10): estado ** OK**, preenchimento 95%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090234_001' (2026-06-12): estado ** OK**, preenchimento 98%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090254_003' (2026-06-12): estado ** OK**, preenchimento 98%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090310_004' (2026-06-12): estado ** Atencao**, preenchimento 95%
 - Produtos desalinados (severidade baixa, local: prateleira 3, centro-direita, area afetada: 5%)
 - Produtos desalinados (severidade baixa, local: prateleira 4, esquerda e centro, area afetada: 10%)
 - Prateleira vazia (severidade baixa, local: prateleira 4, centro, area afetada: 100%)
 - Produtos desalinados (severidade baixa, local: prateleira 5, centro e direita, area afetada: 15%)
 - Prateleira vazia (severidade baixa, local: prateleira 5, canto direito, area afetada: 2%)
 - Outro problema (severidade baixa, local: prateleira 5, varios pontos, area afetada: 100%)
- Inspecao 'INS_20260612_090324_005' (2026-06-12): estado ** OK**, preenchimento 95%
 - Sem problemas detetados.

- Inspecao 'INS_20260612_090340_006' (2026-06-12): estado ** OK**, preenchimento 98%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090359_007' (2026-06-12): estado ** Atencao**, preenchimento 95%
 - Prateleira vazia (severidade baixa, local: prateleira 5, lado esquerdo, area afetada: 15%)
 - Prateleira vazia (severidade baixa, local: prateleira 5, lado direito, area afetada: 10%)
- Inspecao 'INS_20260612_090422_008' (2026-06-12): estado ** Atencao**, preenchimento 92%
 - Prateleira vazia (severidade baixa, local: prateleira inferior, lado direito, area afetada: 5%)
- Inspecao 'INS_20260612_090442_009' (2026-06-12): estado ** Atencao**, preenchimento 92%
 - Produtos desalinados (severidade baixa, local: prateleira superior, porta esquerda, centro, area afetada: 10%)
 - Produtos desalinados (severidade baixa, local: segunda prateleira, porta esquerda, centro/direita, area afetada: 15%)
 - Produtos desalinados (severidade baixa, local: terceira prateleira, porta esquerda, centro/direita, area afetada: 15%)
 - Produtos desalinados (severidade baixa, local: quarta prateleira, porta esquerda, centro/direita, area afetada: 10%)
 - Produtos desalinados (severidade baixa, local: prateleira inferior, porta esquerda, centro/direita, area afetada: 15%)
 - Produtos desalinados (severidade baixa, local: terceira prateleira, porta direita, lado esquerdo, area afetada: 5%)
 - Produtos desalinados (severidade baixa, local: quarta prateleira, porta direita, centro/direita, area afetada: 10%)
 - Outro problema (severidade baixa, local: quarta prateleira, porta direita, centro, area afetada: 5%)
 - Produtos desalinados (severidade baixa, local: prateleira inferior, porta direita, centro/direita, area afetada: 10%)
 - Outro problema (severidade baixa, local: prateleira inferior, porta direita, centro, area afetada: 5%)
- Inspecao 'INS_20260612_090458_010' (2026-06-12): estado ** OK**, preenchimento 98%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090517_011' (2026-06-12): estado ** Atencao**, preenchimento 92%
 - Prateleira vazia (severidade baixa, local: prateleira intermedia superior, lado esquerdo, area afetada: 5%)
 - Produtos desalinados (severidade baixa, local: prateleira intermedia superior, lado esquerdo/centro, area afetada: 2%)
 - Outro problema (severidade baixa, local: prateleira intermedia superior, lado direito, area afetada: 5%)
- Inspecao 'INS_20260612_090538_012' (2026-06-12): estado ** Atencao**, preenchimento 98%
 - Prateleira vazia (severidade baixa, local: prateleira Nivel 5, centro, area afetada: 100%)
- Inspecao 'INS_20260612_090557_013' (2026-06-12): estado ** OK**, preenchimento 97%
 - Sem problemas detetados.
- Inspecao 'INS_20260612_090612_014' (2026-06-12): estado ** Atencao**, preenchimento 88%
 - Prateleira vazia (severidade media, local: prateleira inferior, lado esquerdo/centro, area afetada: 25%)
- Inspecao 'INS_20260612_090627_015' (2026-06-12): estado ** Atencao**, preenchimento 90%
 - Prateleira vazia (severidade media, local: prateleira inferior, seccao centro-direita, area afetada: 35%)
- Inspecao 'INS_20260612_090647_016' (2026-06-12): estado ** Atencao**, preenchimento 95%
 - Produtos desalinados (severidade baixa, local: prateleira 2, lado esquerdo, area afetada: 5%)
 - Produtos desalinados (severidade baixa, local: prateleira 3, centro, area afetada: 100%)
- Inspecao 'INS_20260612_090701_017' (2026-06-12): estado ** OK**, preenchimento 95%
 - Outro problema (severidade baixa, local: prateleira superior intermedia, centro, area afetada: 1%)
- Inspecao 'INS_20260612_090714_018' (2026-06-12): estado ** Atencao**, preenchimento 95%
 - Produtos desalinados (severidade baixa, local:

```
prateleira 3, centro, area afetada: 5%)
- Inspecao 'INS_20260612_090729_020' (2026-06-12): estado **
  OK**, preenchimento 98%
  - Sem problemas detetados.
- Inspecao 'INS_20260612_090740_021' (2026-06-12): estado **
  OK**, preenchimento 95%
  - Sem problemas detetados.
- Inspecao 'INS_20260612_090810_022' (2026-06-12): estado **
  OK**, preenchimento 98%
  - Sem problemas detetados.
- Inspecao 'INS_20260612_090841_023' (2026-06-12): estado **
  Atencao**, preenchimento 95%
  - Prateleira vazia (severidade alta, local: prateleira
    inferior, lado esquerdo, area afetada: 80%)
  - Produtos desalinhados (severidade baixa, local:
    prateleira 4, lado esquerdo, area afetada: 10%)
  - Outro problema (severidade baixa, local: chao, perto da
    prateleira inferior esquerda, area afetada: 0%)
- Inspecao 'INS_20260612_090853_024' (2026-06-12): estado **
  OK**, preenchimento 98%
  - Sem problemas detetados.
- 50 inspecao(oes) sem dados validos (2026-06-12, falha
  temporaria na API):
  'INS_20260612_090236_002', 'INS_20260612_090716_019',
  'INS_20260612_090935_025', 'INS_20260612_091030_026',
  'INS_20260612_091135_027', 'INS_20260612_091216_028',
  'INS_20260612_091255_029', 'INS_20260612_091343_030',
  'INS_20260612_091417_031', 'INS_20260612_091455_032',
  'INS_20260612_091539_033', 'INS_20260612_091621_034',
  'INS_20260612_091719_035', 'INS_20260612_091817_036',
  'INS_20260612_091857_037', 'INS_20260612_091939_038',
  'INS_20260612_092011_039', 'INS_20260612_092048_040',
  'INS_20260612_092120_041', 'INS_20260612_092152_042',
  'INS_20260612_092225_043', 'INS_20260612_092257_044',
  'INS_20260612_092330_045', 'INS_20260612_092403_046',
  'INS_20260612_092441_047', 'INS_20260612_092528_048',
  'INS_20260612_092601_049', 'INS_20260612_092636_050',
  'INS_20260612_092708_051', 'INS_20260612_092741_052',
  'INS_20260612_092813_053', 'INS_20260612_092847_054',
  'INS_20260612_092919_055', 'INS_20260612_092952_056',
  'INS_20260612_093025_057', 'INS_20260612_093058_058',
  'INS_20260612_093130_059', 'INS_20260612_093207_060',
  'INS_20260612_093252_061', 'INS_20260612_093329_062',
  'INS_20260612_093406_063', 'INS_20260612_093440_064',
  'INS_20260612_093515_065', 'INS_20260612_093548_066',
  'INS_20260612_093625_067', 'INS_20260612_093705_068',
  'INS_20260612_093745_069', 'INS_20260612_093819_070',
  'INS_20260612_093854_071', 'INS_20260612_093936_072'

### Zona Z_S2
- Inspecao 'INS_20260610_183302_001' (2026-06-10): estado **
  Atencao**, preenchimento 92%
  - Prateleira vazia (severidade baixa, local: segunda
    prateleira, lado esquerdo, area afetada: 3%)

### Zona Z_S3
- Inspecao 'INS_20260612_113925_001' (2026-06-12): estado **
  OK**, preenchimento 97%
  - Sem problemas detetados.

## 3. Regras Disparadas

- 'RULE_004' [critical] em 'INS_20260612_090841_023' (zona
  Z_S1, 2026-06-12):
  Alerta CRITICO: prateleira vazia com severidade alta
  detetada na zona Z_S1.

## 4. Contexto Historico Relevante

Sem historico relevante para os problemas detetados nesta
sessao.

## 5. Recomendacoes

- [RULE_004] Alerta CRITICO: prateleira vazia com severidade
  alta detetada na
  zona Z_S1.
- Reabastecer com urgencia a zona Z_S2 (prateleira vazia
  detetada).
- Reabastecer com urgencia a zona Z_S1 (prateleira vazia
  detetada).
- Reajustar a arrumacao dos produtos na zona Z_S1 para
  melhorar o
```

```
alinhamento.
- Verificar manualmente o problema assinalado na zona Z_S1.
```

REFERENCES

- [1] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger, and T. Hassner, "Precise Detection in Densely Packed Scenes," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [2] Google DeepMind, "Gemini API Documentation," <https://ai.google.dev/gemini-api/docs>, 2025.
- [3] ChromaDB, "Chroma: the open-source embedding database," <https://www.trychroma.com/>, 2024.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP*, 2019.